

Nelson Ramos

Primera edición



Prompt Engineering

Estrategias de optimización y gestión de IA: Una ruta técnica desde los fundamentos hasta la orquestación de sistemas multiagentes (Edición 2026).

Prompt Engineering para Ingenieros

Cómo reducir costos y automatizar procesos con ecosistemas de agentes

Nelson Ramos

AutoCreativa

2026-03-20

© 2026 Nelson Ramos. Todos los derechos reservados.

Prompt Engineering para Ingenieros

1. [PROMPT ENGINEERING PARA INGENIEROS](#)

1. [Estrategia, Optimización y Despliegue de IA Generativa](#)
2. [Derechos de Autor](#)
3. [Dedicatoria](#)
4. [Sobre el Autor](#)
5. [Prólogo](#)

2. [Capítulo 1: El Costo Oculto de “Solo Usar ChatGPT”](#)

1. [Introducción](#)
2. [1.1 ¿Qué es un Token? La Unidad Económica de la IA](#)
3. [1.2 Anatomía del Pricing: Input vs Output Tokens](#)
4. [1.3 Calculando el Costo Real de una Pipeline de Producción](#)
5. [1.4 El Mapa del Ecosistema 2026: Conoce a Tus Proveedores](#)
6. [1.5 Ejercicio Práctico: Auditoría de Costos en 15 Minutos](#)
7. [Lo que Aprendiste en Este Capítulo](#)
8. [Resumen del Capítulo](#)

3. [Capítulo 2: Cómo Piensa un LLM \(Sin Matemáticas Innecesarias\)](#)

1. [Introducción](#)
2. [2.1 Transformers en 10 Minutos para Ingenieros Ocupados](#)

3. [2.2 Temperatura, Top-P, Top-K: Los Parámetros que Nadie Explica Bien](#)
 4. [2.3 La Ventana de Contexto como Recurso Escaso y Caro](#)
 5. [2.4 Por Qué Más Contexto \$\neq\$ Mejor Respuesta \(y Más Plata Gastada\)](#)
 6. [2.5 Benchmarks 2026: Qué Miden Realmente](#)
 7. [Lo que Aprendiste en Este Capítulo](#)
 8. [Resumen del Capítulo](#)
4. [Capítulo 3: Setup del Entorno de Trabajo Profesional](#)
 1. [Introducción](#)
 2. [3.1 Estructura de Proyecto: Monorepo para Pipelines de IA](#)
 3. [3.2 Manejo de API Keys, Rate Limits y Retry Logic](#)
 4. [3.3 Logging y Observabilidad desde el Día 1](#)
 5. [3.4 Variables de Entorno y Configuración Multi-Modelo](#)
 6. [3.5 Ejercicio Práctico: Scaffolding con Python + litellm](#)
 7. [Lo que Aprendiste en Este Capítulo](#)
 8. [Resumen del Capítulo](#)
5. [Capítulo 4: Prompt Engineering Orientado a Eficiencia](#)
 1. [Introducción](#)
 2. [4.1 El Principio de Mínima Instrucción Efectiva](#)
 3. [4.2 Zero-Shot vs Few-Shot: Cuando Cada Uno Vale la Pena](#)
 4. [4.3 Técnica de Compresión Semántica: Más Instrucción, Menos Tokens](#)
 5. [4.4 Formatos de Output Estructurado: JSON Mode, XML y Markdown](#)

6. [4.5 System Prompts Reutilizables como Activos de Ingeniería](#)
7. [4.6 Ejercicio Práctico: Reducir un Prompt en 60% sin Perder Calidad](#)
8. [4.7 El Salvavidas Asíncrono: Batch API](#)
9. [Lo que Aprendiste en Este Capítulo](#)
10. [Resumen del Capítulo](#)

6. [Capítulo 5: Chain of Thought y Razonamiento Controlado](#)

1. [Introducción](#)
2. [5.1 Cuando el Razonamiento es Caro pero Necesario](#)
3. [5.2 Chain of Thought Estándar vs. Zero-Shot CoT](#)
4. [5.3 Tree of Thought: Ramificación Estratégica](#)
5. [5.4 Modelos de Razonamiento 2026: La Nueva Generación](#)
6. [5.5 La Trampa del “Overthinking”: Cuando Desactivar el Razonamiento](#)
7. [5.6 Ejercicio Práctico: Benchmark de CoT vs Direct Answer](#)
8. [Lo que Aprendiste en Este Capítulo](#)
9. [Resumen del Capítulo](#)

7. [Capítulo 6: Prompt Caching — El Arma Secreta del Ahorro](#)

1. [Introducción](#)
2. [6.1 Cómo Funciona el Caching de Prefijo](#)
3. [6.2 Implementación en Anthropic](#)
4. [6.3 Diseño de Prompts para Maximizar Cache Hits](#)
5. [6.4 Cálculo de ROI del Caching en Producción](#)
6. [6.5 Ejercicio Práctico: Mide el Ahorro Real](#)

7. [Lo que Aprendiste en Este Capítulo](#)

8. [Resumen del Capítulo](#)

8. [Capítulo 7: Selección Dinámica de Modelos \(Model Routing\)](#)

1. [Introducción](#)

2. [7.1 El Principio: No Todo Necesita Claude Opus 4.6](#)

3. [7.2 Clasificadores de Dificultad de Tareas](#)

4. [7.3 Sistema de Routing Completo en Producción](#)

5. [7.4 LLM como Juez: Validación Cruzada entre Modelos](#)

6. [7.5 Ejercicio Práctico: Router con 3 Niveles en tu Proyecto](#)

7. [Lo que Aprendiste en Este Capítulo](#)

8. [Resumen del Capítulo](#)

9. [Capítulo 8: Fundamentos de Agentes de IA](#)

1. [Introducción](#)

2. [8.1 ¿Qué es un Agente? Definición Técnica vs Marketing](#)

3. [8.2 El Ciclo Perceive → Think → Act → Observe](#)

4. [8.3 Tipos de Agentes: ReAct, Plan-and-Execute, Reflexión](#)

5. [8.4 Herramientas \(Tools\): El Lenguaje del Agente con el Mundo](#)

6. [8.5 Memoria en Agentes: Corto y Largo Plazo](#)

7. [8.6 El Problema de la Alucinación en Agentes y Cómo Mitigarlo](#)

8. [Lo que Aprendiste en Este Capítulo](#)

9. [Resumen del Capítulo](#)

10. [Capítulo 9: Diseño de Arquitecturas Multi-Agente](#)

1. [Introducción](#)

2. [9.1 Patrón Orquestador-Subagentes: El Enfoque Más Escalable](#)

3. [9.2 Agentes Especializados vs. Generalistas](#)
 4. [9.3 Comunicación entre Agentes](#)
 5. [9.4 Paralelismo y Concurrency](#)
 6. [9.5 Manejo de Errores en Ecosistemas Distribuidos](#)
 7. [9.6 Patrones Arquitectónicos 2026](#)
 8. [Lo que Aprendiste en Este Capítulo](#)
 9. [Resumen del Capítulo](#)
-
11. [Capítulo 10: Frameworks de Agentes — Guía Comparativa 2026](#)
 1. [Introducción](#)
 2. [10.1 □ OpenClaw: El Framework del Año](#)
 3. [10.2 LangChain / LangGraph](#)
 4. [10.3 CrewAI: Definición Declarativa de Roles](#)
 5. [10.4 Microsoft AutoGen](#)
 6. [10.5 Tabla Comparativa Final](#)
 7. [Lo que Aprendiste en Este Capítulo](#)
 8. [Resumen del Capítulo](#)
-
12. [Capítulo 11: Memoria y Contexto en Ecosistemas de Agentes](#)
 1. [Introducción](#)
 2. [11.1 Tipos de Memoria en Arquitecturas de Agentes](#)
 3. [11.2 Técnicas de Compresión de Contexto](#)
 4. [11.3 RAG \(Retrieval-Augmented Generation\) para Memoria Episódica](#)
 5. [11.4 Compartir Estado Entre Agentes \(Orquestador\)](#)
 6. [Lo que Aprendiste en Este Capítulo](#)

7. [Resumen del Capítulo](#)
13. [Capítulo 12: Herramientas \(Tools\) — Conectando Agentes al Mundo Real](#)
 1. [Introducción](#)
 2. [12.1 ¿Cómo Funciona ‘Tool Use’ Bajo el Capó?](#)
 3. [12.2 Diseño Defensivo de APIs para Agentes](#)
 4. [12.3 El Schema de la Herramienta \(El Contrato con el LLM\)](#)
 5. [12.4 Ejecución de Código como Herramienta Segura \(Code Interpreter\)](#)
 6. [12.5 Forzando el uso de Herramientas \(Tool Choice\)](#)
 7. [Lo que Aprendiste en Este Capítulo](#)
 8. [Resumen del Capítulo](#)
14. [Capítulo 13: Arquitectura de Costos y Control Presupuestario](#)
 1. [Introducción](#)
 2. [13.1 Patrones Anti-Costo en Producción](#)
 3. [13.2 Estrategia de Caching Semántico con Redis / LangChain](#)
 4. [13.3 Monitoreo Financiero y Tokens por Proyecto \(Chargebacks\)](#)
 5. [13.4 Presupuestos “Fail-Safe” Mensuales en Proveedores](#)
 6. [13.5 Tablero Mágico de Equivalencias CLP 2026](#)
 7. [Lo que Aprendiste en Este Capítulo](#)
 8. [Resumen del Capítulo](#)
15. [Capítulo 14: Observabilidad y Monitoreo de Agentes](#)
 1. [Introducción](#)
 2. [14.1 Tracing.\(Traza de Ejecución\) vs Logging Tradicional](#)

3. [14.2 Integrando Langfuse en Python \(Avanzado\)](#)
 4. [14.3 Detectando “Silencios Radiactivos” \(Timeouts de APIs\)](#)
 5. [14.4 Exportación de Datos para Fine-Tuning Futuro](#)
 6. [Lo que Aprendiste en Este Capítulo](#)
 7. [Resumen del Capítulo](#)
16. [Capítulo 15: Evaluación \(Evals\) Automatizada](#)
1. [Introducción](#)
 2. [15.1 ¿Qué es una “Eval” en el Mundo de los LLMs?](#)
 3. [15.2 Construyendo un Framework de Evals Local \(En 30 líneas de Python\)](#)
 4. [15.3 Criterios de Evaluación Comunes](#)
 5. [15.4 Herramientas Gratuitas de 2026 para no inventar la rueda](#)
 6. [15.5 Integrando Prompt Regression Testing en CI/CD](#)
 7. [Lo que Aprendiste en Este Capítulo](#)
 8. [Resumen del Capítulo](#)
17. [Capítulo 16: Casos de Uso — Agentes para Codificación Asistida en tu Terminal](#)
1. [Introducción](#)
 2. [16.1 Agentes de Terminal 2026 \(DevTools de IA\)](#)
 3. [16.2 Diseñando un Agente de Refactoring Personal](#)
 4. [16.3 Generación Dinámica de Tests de Integración](#)
 5. [Lo que Aprendiste en Este Capítulo](#)
 6. [Resumen del Capítulo](#)
18. [Capítulo 17: Análisis de Datos de Alta Complejidad \(RAG 2.0\)](#)

1. [Introducción](#)
2. [17.1 Por qué el RAG Clásico Falla en Producción](#)
3. [17.2 Solución 1: Contextual Retrieval \(El Estándar 2026\)](#)
4. [17.3 Solución 2: Graph RAG \(Grafos de Conocimiento\)](#)
5. [17.4 Agentic RAG: El Agente Multi-Ruta \(Router RAG\)](#)
6. [Lo que Aprendiste en Este Capítulo](#)
7. [Resumen del Capítulo](#)

19. [Capítulo 18: Atención al Cliente y Resolución Autónoma](#)

1. [Introducción](#)
2. [18.1 Nivel 1, Nivel 2 y Nivel 3 en CS \(Customer Success\)](#)
3. [18.2 Manejando el Flujo: The “Unhappy Path”](#)
4. [18.3 Handoff \(Derivación\) al Humano con Resumen Ejecutivo](#)
5. [18.4 Tone Matching \(Espejado Emocional\)](#)
6. [Lo que Aprendiste en Este Capítulo](#)
7. [Resumen del Capítulo](#)

20. [Capítulo 19: Seguridad, Ética y Gobierno \(Guardrails\)](#)

1. [Introducción](#)
2. [19.1 Prompt Injection y Jailbreaks](#)
3. [19.2 Las “Llamadas Fantasma” \(System Prompt Isolation\)](#)
4. [19.3 LNeMo Guardrails y Enfoques “Defensive AI”](#)
5. [19.4 Filtrado de PII \(Información de Identificación Personal\)](#)
6. [Lo que Aprendiste en Este Capítulo](#)
7. [Resumen del Capítulo](#)

21. [Capítulo 20: Despliegue en Producción y Ciclo de Vida \(LLMOps\)](#)

1. [Introducción](#)
2. [20.1 Empaquetado de Orquestadores Multi-Agentes](#)
3. [20.2 Estrategias de Fallback Multicloud \(Alta Disponibilidad\)](#)
4. [20.3 Latency \(Milisegundos\).y Streaming](#)
5. [20.4 Actualizaciones Continuas y Versionado de Prompts](#)
6. [20.5 El “Definition of Done” \(DoD\) para Agentes en Producción](#)
7. [Lo que Aprendiste en Este Capítulo](#)
8. [Resumen del Capítulo](#)

22. [Capítulo 21: El Futuro del Ingeniero Prompt \(2026-2030\)](#)

1. [Introducción](#)
2. [21.1 De “Prompt Engineer” a “AI Systems Engineer”](#)
3. [21.2 Tendencias 2026-2030: Lo que Viene](#)
4. [21.3 Sobreviviendo a la Singularidad \(Profesional\)](#)
5. [Palabras Finales](#)
6. [Lo que Aprendiste en Este Libro](#)

23. [Apéndices del Libro](#)

1. [Apéndice A: Guía Rápida de Costos por Token \(Marzo 2026\)](#)
2. [Apéndice B: Cheatsheet de Parámetros de la API](#)
3. [Apéndice C: Cheatsheet de Frameworks de Agentes](#)
4. [Apéndice D: Referencias y Papers Clave \(Actualizado 2026\)](#)
5. [Apéndice E: Fuentes Oficiales \(Links\) — Marzo 2026](#)
6. [Apéndice F: Glosario de Términos](#)

PROMPT ENGINEERING PARA INGENIEROS

Estrategia, Optimización y Despliegue de IA Generativa

Cómo reducir costos y automatizar procesos con ecosistemas de agentes

Primera Edición — 2026

Autor: Nelson Ramos **Web:** nelsonramos.cl **Emprendimiento:** [AutoCreativa](#) — Talca, Chile

“Este libro nació de la necesidad de reunir la información de internet y darle al usuario un momento y espacio para leer con tranquilidad sobre el estado de la inteligencia artificial a la fecha (marzo de 2026). Fue gestado entre commits, cafés a las 2 AM y la firme certeza de que la IA no va a reemplazarte.”

Derechos de Autor

Copyright © 2026 Nelson Ramos. Todos los derechos reservados.

Publicado a través de Kindle Direct Publishing (Amazon KDP).

Primera edición: Marzo 2026.

Ninguna parte de esta publicación puede ser reproducida, distribuida o transmitida en ninguna forma ni por ningún medio, sin el permiso previo por escrito del autor, excepto en el caso de breves citas incluidas en reseñas críticas y ciertos otros usos no comerciales.

Web del autor: <https://www.nelsonramos.cl> **Empresa:** <https://www.autocreativa.com> **Contacto:** Disponible en el sitio web del autor.

Dedicatoria

A todos los ingenieros e ingenieras de Chile y Latinoamérica que alguna vez abrieron una terminal a las 3 AM preguntándose si el modelo de IA tenía razón o solo estaba alucinando con confianza.

A mi familia, pilar fundamental en este camino, por su infinita paciencia ante mis ideologías, comportamientos y vivencias a través de este proceso. Gracias infinitas.

A mi natal Talca, que me vio volver para reinventarme, y a todas las tazas de café que hicieron posible la compilación de estas páginas.

Sobre el Autor



Nelson Ramos, Desarrollador Full-Stack

Nací en Talca, Chile, el 28 de septiembre de 1990. Desde muy joven desarrollé una curiosidad insaciable y una gran pasión por las ciencias de la computación. Me llamaba mucho la atención aquello que no podía controlar, lo que comúnmente llamamos magia, algo que descubrí en mis primeros contactos con las videoconsolas hacia el año 1998 —específicamente con la Nintendo 64 en casa de mis abuelos—. Recuerdo la sensación de libertad y tranquilidad al poder controlar algo que no hacía daño a través de una televisión; me causó mucha curiosidad saber cómo funcionaba y, desde luego, eso enraizó en mí el interés por la tecnología y la programación. Me titulé como Ingeniero en Ejecución Informática del I.P. Santo Tomás sede Talca en el año 2015. Tras explorar diversas áreas de la informática, en 2019 tomé la decisión definitiva de enfocarme de lleno en el desarrollo de software.

Ese periodo (2019-2020) fue un verdadero punto de inflexión profesional y personal. Durante el estallido social vivía en pleno centro de Santiago, frente al cerro Santa Lucía y a pocas cuadras del metro. Fue una época muy compleja en la que me sentía desarmado frente al sistema, tratando de ganar mi espacio y alegrías. Luego, esto empalmó directamente con la pandemia mundial, lo cual supuso un cambio radical debido al temor que provocó en cada uno de nosotros. Ante estas dificultades, tomé la decisión de volver a Talca, a la casa de mis padres. Aquella mudanza forzada terminó transformándose en una etapa de reinención total: me sumergí de lleno en la formación continua, los bootcamps y el aprendizaje online, abrazando el teletrabajo y el deporte como mi nuevo estilo de vida y comprendiendo la importancia de la salud mental y los hábitos conscientes y positivos. En resumidas cuentas, fue un cambio de *mindset* y *mindfulness* que me otorgó una vida más plena, feliz y consciente.

Quiero agradecer especialmente a plataformas como Platzi por acompañarme en mis horas de estudio matutinas; no puedo olvidar los eventos en vivo donde surge toda esa dinámica de comunidad, y comparto plenamente su lema: “Nunca pares de aprender”. No descarto la posibilidad de asistir como público a una de sus conferencias anuales en el futuro. Todo este profundo proceso de adaptación implicó forjar desde cero nuevos hábitos de estudio y una disciplina profesional inquebrantable.

En el ámbito laboral, he tenido el enorme privilegio de participar en proyectos tecnológicos de gran envergadura a nivel nacional en Chile, colaborando en soluciones para la **Bolsa de comercio de Santiago, Gasconnet, RedPay** y proyectos asociados a **Indra** y **Subtel**. Estas experiencias de alto impacto me permitieron consolidar conocimientos críticos en modularización de software, análisis profundo de sistemas, diseño de infraestructura basada en código (IaC) y arquitectura avanzada de soluciones tecnológicas.

Desde el año 2022 opero al 100% en modalidad de teletrabajo. Paralelamente, me desempeño como freelance a través de mi emprendimiento tecnológico, [AutoCreativa](#). Desde allí, me dedico intensamente a crear proyectos y productos digitales de manera constante, aplicando enfoques de desarrollo modernos como el *vibe coding* y la integración de arquitecturas generativas de IA.

Si deseas conocer más detalles sobre mi trayectoria o necesitas contactarme para colaborar, te invito a visitar mi portafolio profesional en www.nelsonramos.cl o explorar los proyectos en los que trabajo a través de mi emprendimiento en www.autocreativa.com.

Prólogo

Era un martes a la medianoche. Tres tazas de café sobre el escritorio, la cuarta a medio tomar, y yo mirando con espanto una factura de AWS que incluía \$890 USD (~\$860.000 CLP) en llamadas a las APIs de IA durante el mes anterior.

Proyecto de mediana escala. Pipeline de análisis de documentos. Íbamos bien.

O eso creía.

El problema no era el modelo. El problema era yo — o más precisamente, la forma en que yo le estaba hablando al modelo. Prompts redundantes, contextos innecesariamente largos, el mismo system prompt de 2.000 tokens enviado en cada llamada individual sin ningún tipo de caching. Básicamente, estaba tirando dinero al hoyo con cada request.

Fue esa noche cuando empecé a tomar notas. No para escribir un libro — solo para no repetir los mismos errores.

Pero las notas se fueron acumulando. Proyectos en AutoCreativa, experimentos personales, investigación de papers y extensas conversaciones con otros ingenieros de Chile y de toda Latinoamérica que enfrentaban exactamente los mismos problemas. Al final, esas notas se estructuraron y se convirtieron en este libro.

¿Para quién es este libro?

Para el ingeniero o ingeniera en informática que ya sabe programar. Que ya usó ChatGPT, Claude o Gemini. Que ya integró alguna API de IA en un proyecto. Pero que ahora quiere ir al siguiente nivel: diseñar sistemas que sean eficientes, económicos y realmente autónomos.

No es un tutorial para principiantes. No te voy a explicar qué es Python. Asumo que sabes lo que es un endpoint REST, que has debugueado código a medianoche y que el terminal es tu segunda casa.

Lo que vas a aprender:

Este libro cubre tres grandes áreas:

1. **Optimización de costos con Prompt Engineering** — Cómo reducir hasta 70-80% tus gastos en APIs de IA sin sacrificar calidad, usando técnicas comprobadas de la industria.
2. **Ecosistemas de Agentes Automatizados** — Cómo diseñar, implementar y operar redes de agentes de IA que trabajen en paralelo, se comuniquen entre sí y resuelvan problemas complejos de forma autónoma.
3. **Implementación Real** — No teoría vacía. Código, arquitecturas, herramientas y proyectos completos que puedes adaptar desde el día uno.

Una nota sobre los precios:

En este libro vas a encontrar precios tanto en **pesos chilenos (CLP)** como en **dólares estadounidenses (USD)**, porque escribo desde Chile pero sé que me lees desde toda la región. El tipo de cambio de referencia que uso es aproximadamente **\$960 CLP por 1 USD** (valor de referencia a marzo 2026 — siempre verifica el valor actual).

Sobre la velocidad de la industria:

La IA avanza rápido. *Muy* rápido. Mientras escribo esto, hay modelos nuevos saliendo cada mes. Por eso, este libro se enfoca en principios y

patrones que son agnósticos al modelo específico, más que en las características de una versión puntual. Los ejemplos usan los mejores modelos disponibles a principios de 2026, pero las técnicas que aprenderás son aplicables a cualquier LLM, ahora y en el futuro.

Ahora, abre una terminal, prepara un café, y empecemos.

Nelson Ramos

Talca, Chile — Marzo 2026 —

“No se trata de hablarle bien a la máquina. Se trata de diseñar el sistema correcto: aquel donde el lenguaje natural se convierte en la infraestructura que orchestra el futuro.”

Capítulo 1: El Costo Oculto de “Solo Usar ChatGPT”

“El primer proyecto de IA que supervisé en AutoCreativa casi nos cuesta la mitad del presupuesto mensual — y no por el desarrollo, sino por los tokens.” — Nelson Ramos

Introducción

Imagina que contratas a un empleado brillante, capaz de hacer cualquier tarea que le pidas. Pero tiene una condición: cobra por cada palabra que le dices Y por cada palabra que te responde. Además, si repites las mismas instrucciones en cada conversación, pagas de nuevo aunque ya las haya escuchado mil veces antes.

Así funciona una API de lenguaje grande (LLM). Y así es como la mayoría de los equipos de ingeniería, sin darse cuenta, termina gastando de 3 a 5 veces más de lo necesario.

Este capítulo te va a mostrar exactamente dónde se va el dinero — y cómo auditarlo en 15 minutos.

1.1 ¿Qué es un Token? La Unidad Económica de la IA

Antes de hablar de costos, necesitas entender la moneda con que se mide todo en el mundo de los LLMs: el **token**.

Un token no es exactamente una palabra. Es una unidad de texto que el modelo procesa, y que puede ser:

- Una palabra completa: `"hola"` → 1 token
- Una sílaba de una palabra larga: `"automatización"` → aproximadamente 4-5 tokens
- Un espacio o puntuación: `" "` → 1 token
- Código o caracteres especiales: pueden ser más costosos

Regla práctica: En español, 1.000 tokens equivalen aproximadamente a 750 palabras. En inglés la relación es más eficiente: 1.000 tokens $\approx$ 850 palabras. Sí, escribir tus prompts en inglés puede ser más barato.

CÓMO FUNCIONA LA TOKENIZACIÓN DE IA: PROCESO Y COSTOS

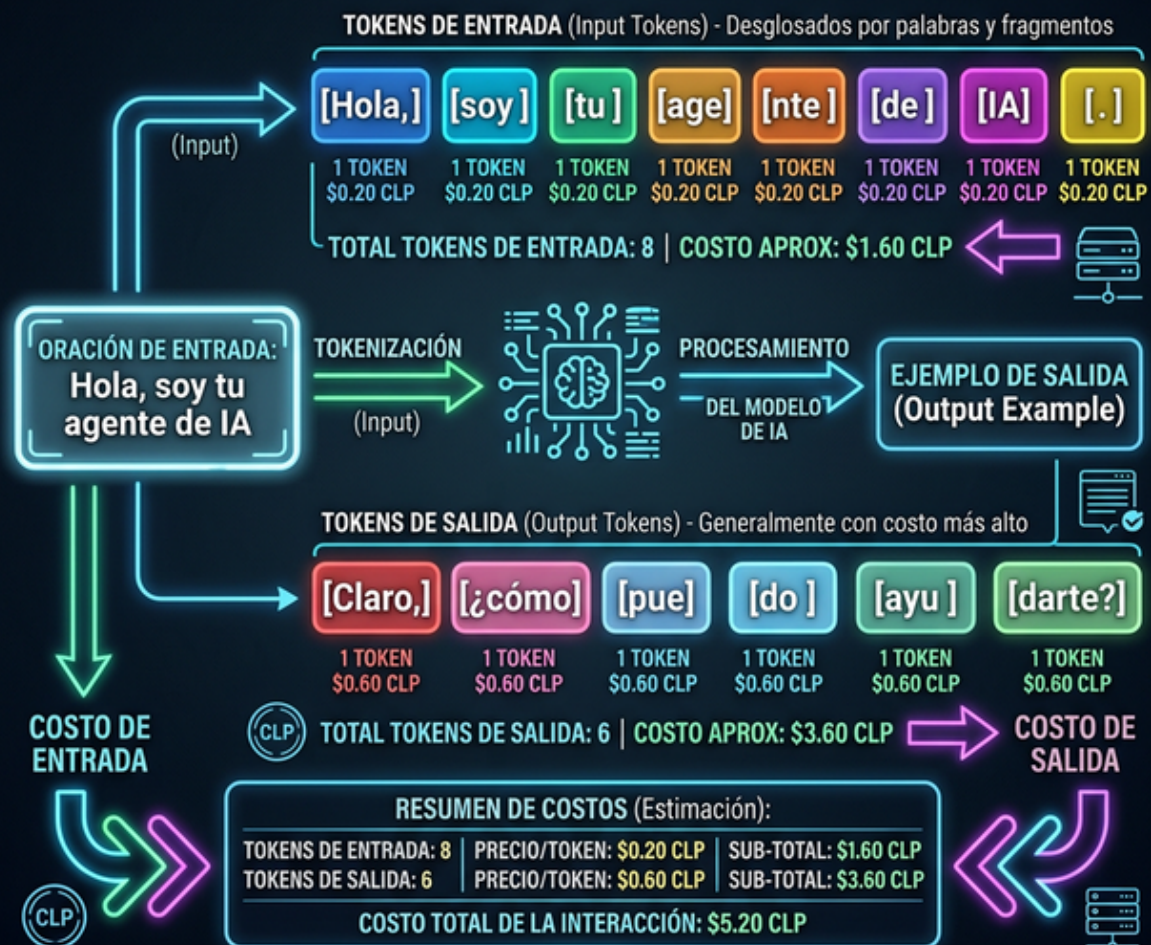


Figura 1.1: El proceso de tokenización convierte texto humano en las piezas fundamentales que el modelo procesa. Nota cómo una sola palabra larga puede dividirse.

El Tokenizador en Acción

Puedes ver exactamente cómo se tokeniza tu texto usando la herramienta de OpenAI en platform.openai.com/tokenizer. Es una de las primeras cosas que deberías revisar cuando optimizas prompts.

Ejemplo práctico con frase en español:

```
"Analiza el siguiente contrato y extrae las cláusulas de penalización"  
= 12 tokens (estimado)
```

```
"Analyze the following contract and extract penalty clauses"  
= 9 tokens (mismo significado, 25% menos tokens)
```

Este pequeño detalle a escala de millones de llamadas se convierte en una diferencia enorme de dinero.

📌 DATO FREAK #1

*El tokenizador de GPT-4 fue entrenado con el algoritmo **Byte Pair Encoding (BPE)**. Si escribes “aaaaaaaaa” (9 ‘a’ seguidas), el modelo lo tokeniza como 1 solo token. Pero si escribes “aaaaaaaaab” (con una ‘b’ al final), ya son 2 tokens. La eficiencia del tokenizado depende de los patrones que el modelo vio durante el entrenamiento — y el español, por ser un idioma con palabras más largas, siempre es un poco menos eficiente que el inglés.*

1.2 Anatomía del Pricing: Input vs Output Tokens

Aquí viene el detalle que muchos ingenieros pasan por alto cuando presupuestan un proyecto de IA: **los tokens de entrada (input) y los de salida (output) tienen precios diferentes**, y el output casi siempre es más caro.

El esquema típico de precios (Marzo 2026)

Nota: Los valores mencionados corresponden a referencias actualizadas a marzo de 2026 y pueden variar con el tiempo. Fuentes oficiales (consulta Mar 2026):

- OpenAI API Pricing: <https://platform.openai.com/docs/pricing> (y también <https://openai.com/api/pricing>)
- Anthropic Claude API Pricing: <https://docs.anthropic.com/en/about-claude/pricing>
- Google Vertex AI (Gemini) Pricing: <https://docs.cloud.google.com/vertex-ai/generative-ai/pricing>

Modelo	Input	Output
Claude Opus 4.6	USD \$5/1M tokens	USD \$25/1M tokens
Claude Sonnet 4.6	USD \$3/1M tokens	USD \$15/1M tokens
Claude Haiku 4.5	USD \$1/1M tokens	USD \$5/1M tokens
GPT-5.4	USD \$2.50/1M tokens	USD \$15/1M tokens

Modelo	Input	Output
gpt-5-mini	USD \$0.25/1M tokens	USD \$2.00/1M tokens
Gemini 3.1 Flash-Lite	USD \$0.25/1M tokens	USD \$1.50/1M tokens

En pesos chilenos (al cambio de \$960 CLP/USD): - Claude Opus 4.6 output: \$24.000 CLP por millón de tokens de respuesta - Gemini 3.1 Flash-Lite output: \$1.440 CLP por millón de tokens — 16 veces más barato

Esto tiene una implicación directa en cómo debes diseñar tus sistemas: **si el modelo habla mucho, pagas mucho**. Por eso el control del output es parte fundamental del Prompt Engineering.

Por qué el output es más caro

Los tokens de salida son más costosos porque el modelo los genera de forma autoregresiva: cada token nuevo requiere una pasada completa de la red neuronal. El input, en cambio, se procesa en paralelo mediante el mecanismo de atención. Generación secuencial vs procesamiento paralelo — ahí está la diferencia de precio.



Figura 1.2: Metáfora visual de la diferencia de costos. Los tokens de entrada (verde) se procesan rápidamente en paralelo como un embudo masivo. Los tokens de salida (dorado) son forjados uno por uno de forma secuencial, por lo que su costo y tiempo de generación son exponencialmente mayores.

1.3 Calculando el Costo Real de una Pipeline de Producción

El error más común es calcular el costo basándose solo en pruebas manuales con ChatGPT. La producción es un animal completamente diferente.

El pipeline que nadie presupuesta bien

Imagina un sistema de análisis de documentos para una empresa chilena. El flujo típico:



Cálculo de costo por documento con Claude Sonnet 4.6: - Input total: 4.000 tokens × \$3/1M = **\$0.012 USD (~\$11.5 CLP)** - Output total: 700 tokens × \$15/1M = **\$0.0105 USD (~\$10 CLP)** - **Total por documento: ~\$0.023 USD (~\$22 CLP)**

Parece barato, ¿verdad? Ahora escala:

Documentos/mes	Costo mensual (USD)	Costo mensual (CLP)
1.000	\$23	~\$22.000
10.000	\$230	~\$220.000
100.000	\$2.300	~\$2.200.000
500.000	\$11.500	~\$11.000.000

Y eso con un modelo mediano como Claude Sonnet 4.6. Si hubieran elegido Claude Opus 4.6 como modelo principal (un error clásico), el costo se **multiplica casi por 1.7**.

☐ DATO FREAK #2

*Según un análisis de Andreessen Horowitz (a16z) de 2025, las empresas que despliegan LLMs en producción gastan en promedio **4,3 veces más de lo presupuestado** en sus primeros 6 meses. El principal culpable: prompts que no estaban optimizados para producción y selección incorrecta de modelos.*

1.4 El Mapa del Ecosistema 2026: Conoce a Tus Proveedores

Para tomar buenas decisiones de costo, necesitas conocer el terreno. A marzo de 2026, los principales jugadores son:

OpenAI (Familia GPT-5.4)

La empresa más conocida. Tiene modelos para todos los presupuestos: - **gpt-5-mini** (USD \$0.25 / \$2.00): ultrabarato, el punto ideal entre costo y calidad para clasificación - **GPT-5.4** (USD \$2.50 / \$15.00): uso general de alta calidad para tareas más exigentes - **o1 / GPT-5.4 Pro** (USD \$15.00 / \$60.00): el más poderoso en razonamiento avanzado — úsalo con cuidado

Anthropic (Claude 4.x Family)

El proveedor preferido de muchos ingenieros para agentes y razonamiento complejo: - **Claude Haiku 4.5**: el más barato de la familia, ideal para tareas simples - **Claude Sonnet 4.6**: el equilibrio perfecto — lo que más uso en proyectos de AutoCreativa - **Claude Opus 4.6**: el más poderoso, con ventana de contexto **muy grande** (según plan/disponibilidad; verifica el límite actual en documentación oficial) para las tareas más exigentes

Google (Familia Gemini 3.1)

Un jugador serio con precios muy competitivos y contexto masivo: - **Gemini 3.1 Flash-Lite** (USD \$0.25 / \$1.50): el más barato y rápido con buena calidad, ideal para alto volumen - **Gemini 3.1 Pro** (USD \$2.00 / \$18.00): comparable a Claude Sonnet 4.6 pero con contexto de hasta 2M tokens de serie

Modelos Locales (Ollama + Llama 4, Mistral, etc.)

El costo cero por token tiene un precio: infraestructura propia. Pero para: - Datos sensibles que no pueden salir de tu servidor - Volúmenes muy altos donde el costo de nube supera el de hardware - Experimentación y desarrollo sin límite de presupuesto

...los modelos locales son la respuesta. Con Ollama en macOS instalas un modelo en 2 comandos.

```
# Instalar Ollama  
brew install ollama  
  
# Descargar y correr Llama 4 en local  
ollama run llama4
```

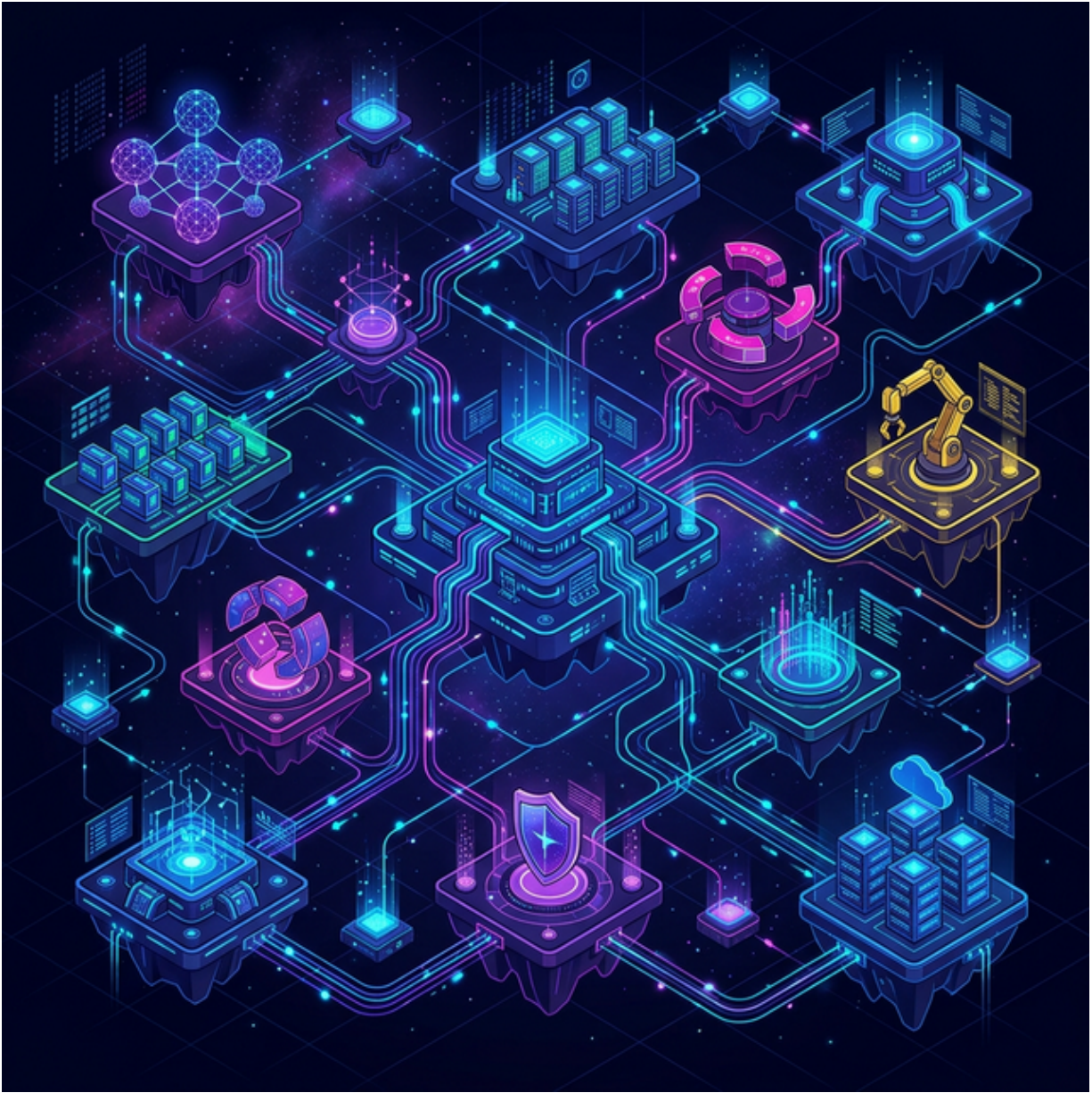


Figura 1.3: Mapa conceptual del ecosistema de IA en 2026. Los modelos comerciales ofrecen conectividad masiva y alta velocidad, mientras que los nodos locales proporcionan entornos aislados y experimentación soberana sin costos de transacción diarios.

1.5 Ejercicio Práctico: Auditoría de Costos en 15 Minutos

Vamos a lo concreto. Si ya tienes un sistema usando APIs de LLM, aquí está cómo auditarlo en 15 minutos.

Paso 1: Revisar el dashboard de tu proveedor (5 min)

Todos los proveedores tienen un panel de uso: - OpenAI: platform.openai.com/usage - Anthropic: console.anthropic.com - Google: console.cloud.google.com

Busca: tokens de input vs output por día, modelos más usados, endpoints más llamados.

Paso 2: Calcula tu ratio input/output (5 min)

```
# Script rápido para analizar logs de costos
import json
from collections import defaultdict

def analizar_costos(logs_path: str):
    """Analiza logs de llamadas a LLM y calcula métricas de costo."""
    stats = defaultdict(lambda: {'input_tokens': 0, 'output_tokens': 0, 'llamadas': 0})

    with open(logs_path) as f:
        for linea in f:
            log = json.loads(linea)
            modelo = log.get('model', 'desconocido')
            stats[modelo]['input_tokens'] += log.get('usage', {}).get('input_tokens', 0)
            stats[modelo]['output_tokens'] += log.get('usage', {}).get('output_tokens', 0)
            stats[modelo]['llamadas'] += 1
```

```

for modelo, data in stats.items():
    ratio = data['output_tokens'] / max(data['input_tokens'], 1)
    print(f"\n❑ Modelo: {modelo}")
    print(f"    Llamadas: {data['llamadas']:,}")
    print(f"    Input tokens: {data['input_tokens']:,}")
    print(f"    Output tokens: {data['output_tokens']:,}")
    print(f"    Ratio output/input: {ratio:.2f}x")
    if ratio > 1.0:
        print(f"    ⚠ Output mayor que input – revisa si el modelo sobre-genera")

analizar_costos('api_logs.jsonl')

```

Paso 3: Identifica las 3 llamadas más costosas (5 min)

El principio de Pareto aplica aquí: el 20% de tus llamadas probablemente representa el 80% de tu costo. Encuéntralas y optimízalas primero.

Señales de alerta a buscar: - ❑ System prompts > 1.000 tokens enviados en *cada* llamada (candidato a caching) - ❑ Output > 2.000 tokens cuando solo necesitas una respuesta corta (el modelo sobre-genera) - ❑ Múltiples llamadas secuenciales que podrían ser una sola con mejor prompt - ❑ Modelo de alta gama para tareas simples de clasificación o extracción

Lo que Aprendiste en Este Capítulo

Este capítulo estableció la base económica de todo el libro: los tokens son la moneda invisible de la IA, y entender cómo funcionan es la diferencia entre un proyecto rentable y uno que quema presupuesto. Aprendiste que un token no es una palabra completa sino una unidad de procesamiento que varía según el idioma —el español es menos eficiente que el inglés porque tiene palabras más largas—, y que escribir prompts en inglés puede ahorrarte un 25% en tokens.

Comprendiste la asimetría fundamental de costos: los tokens de output son 3 a 5 veces más caros que los de input porque el modelo los genera secuencialmente (uno por uno), mientras que el input se procesa en paralelo. Esto tiene implicaciones directas en el diseño: si tu sistema hace que el modelo “hable mucho”, estás pagando innecesariamente.

Calculaste el costo real de una pipeline de producción, descubriendo que lo que parece barato en pruebas manuales con ChatGPT se multiplica exponencialmente en producción. Conociste el ecosistema de proveedores de 2026: desde Gemini Flash-Lite a \$1.440 CLP por millón de tokens output (el precio de un pasaje de metro) hasta Claude Opus a \$24.000 CLP (el precio de una asesoría profesional). Finalmente, ejecutaste una auditoría de 15 minutos para identificar los tres mayores quemadores de presupuesto: system prompts redundantes, outputs excesivos y modelos de alta gama usados para tareas simples.

Resumen del Capítulo

En este capítulo cubrimos los fundamentos económicos del trabajo con LLMs:

1. **Los tokens son la moneda** — aprende a estimarlos y a favorecer el inglés cuando sea posible
2. **Output es siempre más caro que input** — diseña tus sistemas para generar respuestas concisas
3. **Escala tu cálculo de costo desde el día 1** — lo que parece barato en desarrollo puede ser caro en producción
4. **Conoce el ecosistema** — la diferencia de precio entre modelos puede ser de 15 a 20 veces

5. **Audita antes de optimizar** — tienes 15 minutos para encontrar el mayor problema de costo

En el próximo capítulo, vamos a abrir el capó y entender cómo piensan realmente estos modelos — porque para optimizar algo, primero hay que entender cómo funciona.

*Siguiente: **Capítulo 2 — Cómo Piensa un LLM (sin matemáticas innecesarias)***
